LangSmith — Concepts and Python Implementation

Table of Contents

---

1. What Is LangSmith?

LangSmith is LangChain's platform for **observability, evaluation, prompt management, and deployment** of LLM and agentic applications. The clean mental model from the LangChain team is straightforward: LangChain is the framework for building blocks, LangGraph is the orchestration runtime for anything agentic or multi-step, and LangSmith is the platform that sits on top of both — giving visibility into what happened, measuring whether it was good, and running it in production. [1]

It exists because agentic systems fail in ways plain logging can't surface. A modern agent run is a tree of nested LLM calls, tool invocations, retries, and conditional branches — you cannot debug that with `logger.info`. [2] LangSmith lets teams inspect application runs, traces, and step-level behavior to see how an agent actually behaved instead of guessing from the final output alone, since for agent systems the final answer is often the least informative part of a failure. [3]

Although it originated for LangChain apps, it is framework-agnostic on paper — traces can be sent from any application, and as of 2026 LangSmith ships end-to-end OpenTelemetry support so any OTel-compatible app can export to the LangSmith endpoint. [4] That said, instrumenting a LangGraph agent specifically means the traces, the node graph, the state at each step, and the deploy path all line up with near-zero glue code — non-LangChain/LangGraph apps get a smaller slice of the platform's value. [4]

---

2. Core Capabilities

Capability	What It Gives You
Tracing	Full run tree of every LLM call, tool call, and nested function in an agent run, with latency and token usage per step
Evaluation	Offline datasets + evaluators (heuristic, LLM-as-judge, custom) to score output quality systematically
Prompt management	Versioning and iteration on prompts, decoupled from application code
Production monitoring	Cost, latency, error-rate, and p95 dashboards, with threshold-based alerts before customers notice a regression [5]
Deployment	Hosted deployment via LangGraph Platform [1], plus LangSmith Fleet for agent deployment and a unified cost view across full agent workflows [6]

> **2026 addition:** *LangSmith Engine* — an AI layer that analyzes traces and suggests fixes for failing or expensive runs. [5]

---

3. How Tracing Actually Works

This is the part most people get wrong initially, so it's worth being precise:

- Under the hood, LangSmith uses decorators, context managers, and a run-tree data structure to capture each step in a chain (or any function) as a traceable run. [7]
- **`@traceable` is not automatically "on."** The decorator on its own has no effect unless tracing is actually enabled — either via environment variables or a context manager. [8] This has tripped up enough people that it remains an open, frequently-referenced SDK issue.
- Decorators are zero-cost when tracing is disabled — they return the raw function if `LANGCHAIN_TRACING_V2` is unset, so it's safe to ship `@traceable` decorators in code that runs in environments without tracing configured. [9]
- Overhead in v2-client testing stayed below 4 ms per run, because the SDK buffers traces and flushes them in a background thread rather than blocking the main request path. [9]
- For LangChain-native components (`ChatOpenAI`, chains, etc.), tracing is wired through LangChain's callback system — specifically the `LangChainTracer` callback handler, triggered via `tracing_v2_enabled()` or the same environment variables. [7] `@chain` and `@traceable` are therefore complementary, not interchangeable.

---

4. Setup

4.1 Install

```bash
pip install -U langsmith langchain langgraph
```

4.2 Environment Variables

```bash
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_API_KEY="your-langsmith-api-key"
export LANGCHAIN_PROJECT="enrichment-framework"   # optional — defaults to "def
```

> If `LANGCHAIN_PROJECT` isn't set, traces are logged under a project named `"default"`. [10]

---

5. Tracing in Python

5.1 Minimal Example — Non-LangChain Function

```python
import os
from langsmith import traceable

os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your-langsmith-api-key"
os.environ["LANGCHAIN_PROJECT"] = "enrichment-framework"

@traceable(name="fetch_broker_from_db")
def fetch_broker_from_db(broker_id: str) -> dict:
    # ... your DB lookup logic
    return {"broker_id": broker_id, "name": "Acme Insurance Brokers"}

fetch_broker_from_db("BRK-1029")
```

5.2 Nested Traces — Mapping Directly onto a Multi-Agent Pipeline

Nesting happens automatically when a `@traceable`-decorated function calls another `@traceable`-decorated function — this is exactly the shape of an `enrichment_worker → reflection_agent → synthesizer` pipeline:

```python
```

```python
from langsmith import traceable
from langsmith.wrappers import wrap_openai
from openai import OpenAI

client = wrap_openai(OpenAI())  # nested LLM calls inside are auto-traced

@traceable(name="enrichment_worker", run_type="chain")
def enrichment_worker(attribute_name: str, sources: dict) -> dict:
    evidence = collect_evidence(attribute_name, sources)
    candidate = extract_with_llm(attribute_name, evidence)
    return {"candidate": candidate, "evidence": evidence}

@traceable(name="collect_evidence", run_type="tool")
def collect_evidence(attribute_name: str, sources: dict) -> str:
    # DB + web tool calls here
    return "combined evidence text"

@traceable(name="extract_with_llm", run_type="llm")
def extract_with_llm(attribute_name: str, evidence: str) -> dict:
    response = client.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": f"Extract {attribute_name} from:
    )
    return {"value": response.choices[0].message.content}

@traceable(name="reflection_agent", run_type="chain")
def reflection_agent(candidate: dict, evidence: str) -> dict:
    # confidence scoring + rerouting logic
    return {"confidence": 0.82, "reroute": False}

@traceable(name="synthesizer", run_type="chain")
def synthesizer(candidate: dict, reflection: dict) -> dict:
    return {"final_value": candidate["value"], "confidence": reflection["confid

@traceable(name="enrichment_pipeline_run")
def run_pipeline(attribute_name: str, sources: dict) -> dict:
    worker_out = enrichment_worker(attribute_name, sources)
    reflection_out = reflection_agent(worker_out["candidate"], worker_out["evid
    return synthesizer(worker_out["candidate"], reflection_out)
```

Every call here — from `run_pipeline` down through `extract_with_llm` — appears as one nested run tree in the LangSmith UI, with per-node latency and (for the `run_type="llm"`

node) token usage and cost.

`run_type` **matters.** Setting it to `"llm"`, `"tool"`, `"chain"`, or `"retriever"` controls how the run is categorized and rendered — cost tracking, for example, only applies to `"llm"` runs. LangSmith automatically prices OpenAI, Anthropic, Google, and Azure calls as soon as it sees token usage in a run; if a run shows zero cost, that model likely needs manual pricing configured. [9]

5.3 Adding Metadata and Tags

```python
@traceable(
    name="extract_with_llm",
    run_type="llm",
    tags=["attribute-extraction", "production"],
    metadata={"attribute": "annual_turnover", "model_version": "gpt-4"},
)
def extract_with_llm(attribute_name: str, evidence: str) -> dict:
    ...
```

Metadata and tags are how you later filter and segment traces in the dashboard — e.g. isolating every run for a specific attribute, or comparing the hybrid Pythonic path against the LLM-reasoning path.

5.4 Automatic Tracing for LangGraph

If your enrichment pipeline is a `StateGraph`, tracing requires no extra decoration beyond the environment variables — every node execution, conditional edge decision, and state transition is captured automatically once `LANGCHAIN_TRACING_V2=true` is set, and LangGraph Studio can step through it visually. Integrating with LangGraph also gives persistent memory and human-in-the-loop checkpoints — the two patterns that matter most once you're past prototype stage. [5]

---

6. Evaluation

6.1 Building a Dataset

```python
```

```python
from langsmith import Client

client = Client()

dataset = client.create_dataset(
    dataset_name="enrichment-annual-turnover-eval",
    description="Ground-truth annual_turnover extractions for regression testin
)

examples = [
    {"inputs": {"attribute_name": "annual_turnover", "evidence": "Doc excerpt A
     "outputs": {"expected_value": "$4.2M"}},
    {"inputs": {"attribute_name": "annual_turnover", "evidence": "Doc excerpt B
     "outputs": {"expected_value": "$980K"}},
]

client.create_examples(
    inputs=[e["inputs"] for e in examples],
    outputs=[e["outputs"] for e in examples],
    dataset_id=dataset.id,
)
```

LangSmith's dataset format is opinionated — inputs/outputs as structured dicts — a deliberate trade-off against something like a free-form CSV, in exchange for tighter integration with evaluators and experiment comparison. [11]

6.2 Defining an Evaluator and Running an Experiment

python

```python
from langsmith.evaluation import evaluate

def correctness_evaluator(run, example) -> dict:
    predicted = run.outputs.get("final_value")
    expected = example.outputs.get("expected_value")
    return {"key": "correctness", "score": int(predicted == expected)}

results = evaluate(
    lambda inputs: run_pipeline(inputs["attribute_name"], inputs["evidence"]),
    data="enrichment-annual-turnover-eval",
    evaluators=[correctness_evaluator],
    experiment_prefix="hybrid-resolution-v1",
)
```

This produces a versioned **experiment** you can compare run-over-run in the dashboard — e.g. comparing the current LLM-only enrichment path against the hybrid Pythonic/LLM redesign on the same dataset, to confirm accuracy doesn't regress while latency drops.

6.3 LLM-as-Judge Evaluators

For criteria that can't be checked by exact match (e.g. "is the extracted address plausible given the evidence?"), LangSmith supports LLM-as-judge evaluators:

```python
from langsmith.evaluation import LangChainStringEvaluator

judge = LangChainStringEvaluator(
    "criteria",
    config={"criteria": "Is the extracted value directly supported by the provi
)

results = evaluate(
    lambda inputs: run_pipeline(inputs["attribute_name"], inputs["evidence"]),
    data="enrichment-annual-turnover-eval",
    evaluators=[judge],
)
```

6.4 Online Evaluation (Production Sampling)

Beyond offline experiments, LangSmith supports online evaluators applied to a sample of production traces [11] — e.g. configuring a rule to run the correctness/criteria evaluator against 10% of live enrichment runs, feeding a rolling quality dashboard instead of relying only on pre-deployment test runs.

---

7. Production Monitoring

Once tracing is live in production:

- **Cost, latency, error-rate, and p95 dashboards** come out of the box from run-tree data — no separate instrumentation needed.
- **Alerting.** Configure thresholds per metric (e.g. token-cost spike, elevated error rate), so a 3× spike in tokens-per-call is treated as a signal rather than a coincidence discovered later. [5]
- **Raw access.** `client.list_runs()` lets you pull traces programmatically for custom aggregation outside the dashboard — useful for building your own rollups on top of LangSmith's stored run data. [9]

```python
```

```python
from langsmith import Client

client = Client()

runs = client.list_runs(
    project_name="enrichment-framework",
    filter='eq(name, "reflection_agent")',
    start_time="2026-07-01T00:00:00Z",
)

reroute_count = sum(1 for r in runs if r.outputs.get("reroute") is True)
```

This is exactly how you'd empirically answer "how often is the reflection agent rerouting back to the enrichment worker" — a question that otherwise requires manual log-diving.

---

8. Practical Notes and Trade-offs

1. **Don't over-instrument.** Tracing every internal function call produces noise and inflates cost — trace only the LLM calls and key business-logic functions, not every helper. [11]
2. **Decorators are safe to ship everywhere.** Since `@traceable` is a no-op without `LANGCHAIN_TRACING_V2=true`, there's no need to strip tracing code out of production builds where tracing is off.
3. **Datasets go stale.** A dataset that no longer reflects current product behavior misleads — refresh it periodically (e.g. quarterly) from real production samples. [11]
4. **Calibrate LLM-as-judge evaluators.** If an evaluator isn't checked against human judgments, its scores shouldn't be trusted at face value — spot-check judge verdicts against a human-labeled subset before relying on them for regression gating. [11]
5. **Hosting model.** LangSmith is primarily a managed cloud service; self-hosted/hybrid deployment is offered only on higher (Enterprise) tiers with a custom quote, and reported details vary by source — confirm current self-hosting and data-residency terms directly on LangChain's site before committing to it for compliance-sensitive workloads.
6. **`run_type` drives cost tracking.** Only runs tagged `"llm"` get automatic token-based cost attribution — mislabeling a call as `"chain"` or `"tool"` silently drops it from cost dashboards.

---

9. Summary

Need	LangSmith Feature
See what an agent actually did, step by step	Tracing (`@traceable`, automatic for LangGraph)
Measure output quality systematically	Datasets + offline evaluators/experiments
Catch quality regressions in production	Online evaluators on sampled traces
Iterate on prompts without redeploying code	Prompt Hub / prompt versioning
Know when something breaks or gets expensive	Cost/latency/error dashboards + alerts
Ship an agent, not just prototype it	LangGraph Platform deployment / LangSmith Fleet

For a multi-agent pipeline like an enrichment framework, the highest-leverage starting point is usually:

1. Wrap the three main agent functions in `@traceable` with meaningful `run_type`s and metadata.
2. Build a 30–50 example evaluation dataset from real attribute-extraction cases.
3. Run experiments comparing architecture changes (like the Pythonic/LLM hybrid) against that fixed dataset before rolling changes into production.

---

10. Sources

Note: several sources above are third-party guides rather than official LangChain documentation; treat version-specific details (pricing, self-hosting terms, exact API signatures) as directional and verify against docs.langchain.com before relying on them for production decisions.

1. LangChain vs LangGraph vs LangSmith: What's the Difference in 2026 — truefoundry.com ↩ ↩2
2. How to Use LangSmith in 2026: Complete Tracing & Debugging Guide — ai.cc ↩
3. What Is LangSmith? A Practical 2026 Guide to Tracing, Evals, and Agent Deployment — nerova.ai ↩
4. LangSmith Review 2026: LLM Observability, Tested — aitoolsbakery.com ↩ ↩2
5. How to Use LangSmith in 2026: Complete Tracing & Debugging Guide — ai.cc ↩ ↩2 ↩3 ↩4
6. What is LangSmith? 2026 Guide to LLM Observability — metacto.com ↩

7. LangSmith Tracing Deep Dive — Beyond the Docs — Medium ↩ ↩2
8. How do I enable tracing with the `@traceable` decorator — Issue #1549 — GitHub ↩
9. LangSmith in Production: Tracing, Monitoring & Cost Control — markaicode.com ↩ ↩2 ↩3 ↩4
10. langsmith-cookbook: tracing_without_langchain.ipynb — GitHub ↩
11. LangSmith Evaluation Platform Complete Guide 2026 — qaskills.sh ↩ ↩2 ↩3 ↩4 ↩5